recommended practices when writing consumer contract tests, and we'll cover the following topics:

- The focus of a consumer contract test
- The importance of using mocks instead of stubs
- Using loose matchers to avoid high coupling
- The importance of writing isolated contract tests
- Using can-i-deploy to verify that you can safely deploy to production

4.1.1 The focus of a consumer contract test

Consumer contract tests have one focus—to make sure that when a consumer asks for data from a provider service, everything goes according to their contract agreement. Imagine it as a way to check if the consumer's expectations match what the provider actually does. These tests don't dig into the nitty-gritty functional details of how the provider works; instead, they focus on what the consumer needs and checks if they get it.

Think of consumer contract tests as a security guard at a building entrance where security is high. They make sure that everyone follows the rules laid out in their working contract. A consumer contract test acts as a security guard to catch any mistakes or misunderstandings in how the consumer asks for data and what they expect in return from the provider.

A general rule of thumb that the Pact tool recommends (https://docs.pact.io/consumer) is to ask yourself this question: "If I don't include this scenario, what bug in the consumer or what misunderstanding about how the provider responds might be missed?" If the answer is none, don't include it.

4.1.2 The importance of using mocks instead of stubs

Mocks and stubs are two important techniques developers have used to write a unit test. They act as "test doubles," like a movie stunt double. These two techniques are different, and knowing the difference in contract testing is important.